



Assessing the Potential Impact of Memory-Safe Languages for System Software Security

Sponsor: NSA, Neal Ziring
By: University of Arizona and UT Dallas Team 3



Agenda

- Project Description
- Project Objectives and Goals
- Investigator Biographies
- Research Plan Summary
- Project Deliverables
- Progress to date
- Conclusions to date
- Issues/Challenges
- Bibliography
- Acronyms
- Terms/Definitions



Top Level Research Project Description

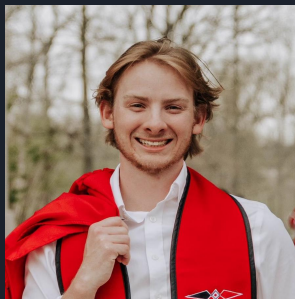
- Assessment of traditional C/C++ memory vulnerabilities in C/C++ and WebAssembly.
- Exploration of memory safety features of Rust and Go.
- Identification of vulnerabilities through CWE analysis.
- Provision of coding examples in multiple languages.
- Testing in WebAssembly for superior memory safety with Rust and Go.
- Integration of AI tools like ChatGPT for effectiveness in vulnerability detection and remediation in source code.



Project Objectives / Goals

- Examination of the viability of conventional migration strategies for addressing vulnerabilities in WebAssembly.
- Emphasis on the security advantages of Rust and Go.
- Inclusion of case studies showcasing real-world applications.
- Illustration of successful vulnerability mitigation in the WebAssembly context.
- Culmination of findings in a presentation of key insights and recommendations.
- Advocacy for the adoption of memory-safe languages in security-sensitive projects.
- Aim to provide a well-rounded perspective on enhancing security in WebAssembly development.

Investigator Biographies



Joshua McCoy

Education:

- B.S. in Computer Science, Miami University, Oxford, Ohio.
- Graduate Student in Cybersecurity, University of Arizona.

Experience:

- Systems Security Engineer at RTX, Tucson, AZ, focusing on PIT System Cybersecurity and Anti-Tamper for missile systems.
- Software Engineering intern at Microsoft (React.js).
- Data Science intern at The Center for Analytics and Data Science (Data Analytics, R, RStudio).
- Software Developer for The Miami Tribe of Oklahoma (HTML, CSS, JavaScript, SQL).
- Associate Developer Consultant at Cyborg Mobile (React.js).



Chia-Hao Yang

Education:

- B.S. in Electrical Engineering.
- M.S. in Computer Science (in progress) at the University of Texas at Dallas.
- Graduate Certificate in Cyber Defense (NSA CAE).

Experience:

- Top 10% in NSA Codebreaker Challenge 2022.
- 3rd place in TexSAW 2022 CTF competition.



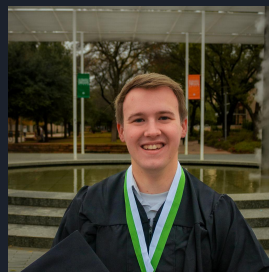
Nate Schultz

Education:

- B.A. in Information Communication Technology and Policy, University of Hawaii.
- M.S. in Management Information Systems, University of Arizona.
- Currently completing a Cybersecurity Master's at the University of Arizona.

Experience:

- 15 years in IT, with the last 3 years as a System Administrator at RTX.



Ryan Brockinton

Education:

- B.S. in Computer Science from the University of Texas at Dallas.
- Pursuing an M.S. in Computer Science with an emphasis in Cybersecurity at the same university.

Experience:

- Participated in Glacier CTF 2023, focusing on reverse engineering, low-level programming, and web exploitation.
- Multiple internships at Texas Instruments, focusing on system automation and web development.



Research Plan Summary

- Research into memory Common Weakness Enumerations (CWEs) in WebAssembly.
- Division of tasks among team members into two groups: programming and research.
- Creation of C/C++ examples to exploit vulnerabilities in both native environments and WebAssembly.
- Comprehensive study of WebAssembly-related cases in the National Vulnerability Database (NVD).
- Implementation of traditional migration strategies in native environments, coupled with a literature review.
- Testing the feasibility of traditional migration strategies in the WebAssembly environment.
- Conclusion of the literature review.
- Recreation of C/C++ code using Rust and Go to validate AI tools' effectiveness in detecting and remediating vulnerabilities.
- Testing Rust and Go code behavior in the WebAssembly environment.
- Statistical analysis of results from different AI tools to understand their effectiveness in securing the system.



Project Deliverables

- A comprehensive final report documenting research methodologies, findings, and insights.
- Professionally crafted presentation materials for academic and industry audiences.
- Establishment of a dedicated repository for transparency and reproducibility.
- A curated collection of test cases in the repository for replicating experiments.
- Analysis tools developed during the project housed in the repository.
- Aim to empower scholars, developers, and cybersecurity practitioners with resources to validate and build upon project outcomes.



Progress to Date

Phase 1: Research

- Looking at previous teams projects
- C/C++ Vulnerabilities
- Researching Vulnerabilities in NVD and CWE databases
- Language Knowledge (whole team is fresh to Go and Rust)
- Literature Reviews



Progress to Date

Phase 2: CWE Selection

- While there are many applicable CWE's to C and C++ not all of these are applicable to WebAssembly as well
- We ended up with 5 total CWE's that we aim to work through in our testing, two of which are “child” vulnerabilities to another overarching one
- CWE-476: NULL Pointer Dereference
- CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer
 - CWE-787: Out-of-bounds Write
 - CWE-121: Stack-based Buffer Overflow
 - CWE-122: Heap-based Buffer Overflow



Progress to Date

CWE-121: Stack-based Buffer Overflow

	Native Environment	WebAssembly Environment
Testing Compiler	GNU Compiler Collection (GCC)	Emscripten Compiler Frontend (EMCC)
Stack memory address	Fixed	Fixed
Detect Stack-based Buffer Overflow	Yes	No

1. WebAssembly environment lacks detection of stack-based buffer overflow.
2. Stack section in webassembly will be clear before written in default.



Progress to Date

CWE-121: Stack-based Buffer Overflow

```
argv[1]: a_20bytes_1234567890
```

Execute:

```
char buf1[16], buf2[16];  
strcpy(buf1, "password");
```

Stack Data 1: password, Memory Address: 0x16b1b3388

Stack Data 2: 04, Memory Address: 0x16b1b3378

Execute:

```
strcpy(buf2, argv[1]);
```

Trace/BPT trap: 5

Native Environment

```
argv[1]: a_20bytes_1234567890
```

Execute:

```
char buf1[16], buf2[16];  
strcpy(buf1, "password");
```

Stack Data 1: password, Memory Address: 0xffa0

Stack Data 2: , Memory Address: 0xff90

Execute:

```
strcpy(buf2, argv[1]);
```

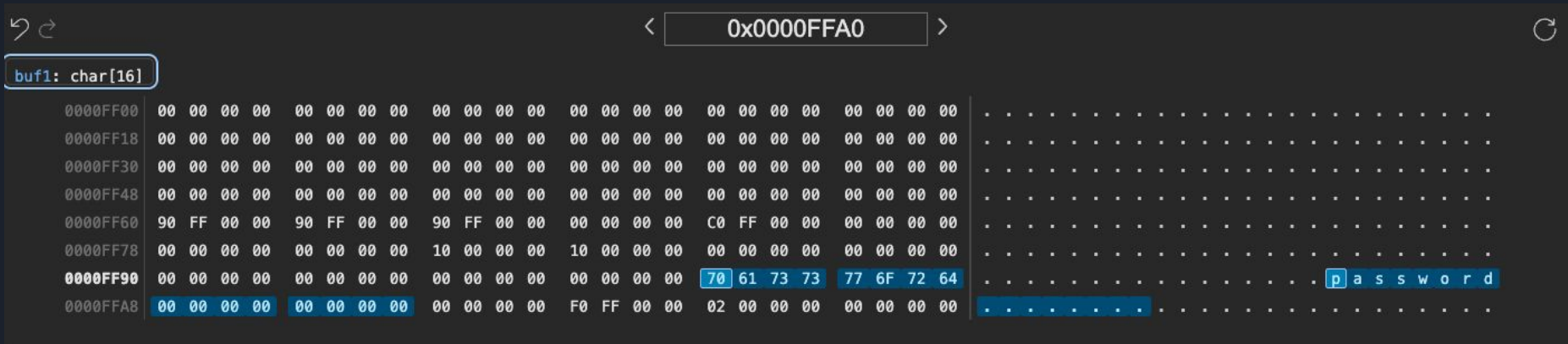
Stack Data 1: 7890, Memory Address: 0xffa0

Stack Data 2: a_20bytes_1234567890, Memory Address: 0xff90

WebAssembly Environment

Progress to Date

CWE-121: Stack-based Buffer Overflow



	<	0x0000FFA0	>	
buf1: char[16]				
0000FF00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0000FF18	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0000FF30	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0000FF48	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0000FF60	90 FF 00 00	90 FF 00 00	90 FF 00 00	C0 FF 00 00
0000FF78	00 00 00 00	00 00 00 00	10 00 00 00	10 00 00 00
0000FF90	00 00 00 00	00 00 00 00	00 00 00 00	70 61 73 73 77 6F 72 64
0000FFA8	00 00 00 00	00 00 00 00	F0 FF 00 00	02 00 00 00

Memory Dynamic Analysis Tool: C/C++ DevTools Support (DWARF)

- Provided by Chrome DevTools Team.
- Available to trace memory usage in browser like GDB in native environment.



Progress to Date

CWE-122: Heap-based Buffer Overflow

	Native Environment	WebAssembly Environment
Testing Compiler	GNU Compiler Collection (GCC)	Emscripten Compiler Frontend (EMCC)
Heap memory address	Randomized	Fixed
Default buffers gap	416 bytes	16 bytes

1. Higher risk of the heap section in WebAssembly being identified with specific memory addresses.
2. Shorter input strings in the heap section of WebAssembly may result in buffer overflow.

Progress to Date

CWE-122: Heap-based Buffer Overflow

```
argv[1]: a_20bytes_1234567890  
argv[2]: b_20bytes_1234567890
```

Execute:

```
char *ptr1, *ptr2;  
ptr1 = malloc(8);  
ptr2 = malloc(8);
```

```
Heap Data 1: , Memory Address: 0x151605e60  
Heap Data 2: , Memory Address: 0x151606000
```

Execute:

```
strcpy(ptr1, argv[1]);
```

```
Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60  
Heap Data 2: , Memory Address: 0x151606000
```

Execute:

```
strcpy(ptr2, argv[2]);
```

```
Heap Data 1: a_20bytes_1234567890, Memory Address: 0x151605e60  
Heap Data 2: b_20bytes_1234567890, Memory Address: 0x151606000
```

Native Environment

```
argv[1]: a_20bytes_1234567890  
argv[2]: b_20bytes_1234567890
```

Execute:

```
char *ptr1, *ptr2;  
ptr1 = malloc(8);  
ptr2 = malloc(8);
```

```
Heap Data 1: , Memory Address: 0x10a88  
Heap Data 2: , Memory Address: 0x10a98
```

Execute:

```
strcpy(ptr1, argv[1]);
```

```
Heap Data 1: a_20bytes_1234567890, Memory Address: 0x10a88  
Heap Data 2: 7890, Memory Address: 0x10a98
```

Execute:

```
strcpy(ptr2, argv[2]);
```

```
Heap Data 1: a_20bytes_123456b_20bytes_1234567890, Memory Address: 0x10a88  
Heap Data 2: b_20bytes_1234567890, Memory Address: 0x10a98
```

WebAssembly Environment



Progress to Date

CWE-476: NULL Pointer Dereference

	Native Environment		WebAssembly Environment
	C	C++	C
Testing Compiler	GNU Compiler Collection (GCC)	GNU Compiler Collection (G++)	Emscripten Compiler Frontend (EMCC)
Resulting Error	Segmentation fault	Undefined behavior based on conditions	Abort

1. C++ is more vulnerable due to the chance that the behavior appearing to be successful
2. Webassembly causes the dereferenced pointer to corrupt the heap unlike in the native environment



Progress to Date

CWE-476: NULL Pointer Dereference

```
8  class SampleClass {
9      int x;
10
11     public:
12         void setX() {
13             x = 10;
14         }
15         void printMessage() {
16             std::cout << "Hello World!" << std::endl;
17         }
18     };
19
20     SampleClass* createSampleClass(char *input) {
21         if (strcmp(input, "valid") == 0) {
22             std::cout << "valid pointer test\n" << std::endl;
23             return new SampleClass();
24         } else {
25             // Simulating a condition that leads to returning a NULL pointer
26             std::cout << "null pointer dereference test\n" << std::endl;
27             return nullptr;
28         }
29     }
```



Summary of Results / Conclusions to Date

- Extensive literature exists on C/C++ memory vulnerabilities and surprisingly there is also a good amount of literature on WebAssembly memory vulnerabilities
- Literature/Research has shown our desire to test WebAssembly alongside C/C++ compared to “Memory Safe” languages is a quite worthy project
- Some CVE’s we found are not applicable in WebAssembly (even if they are in C or C++) so this lead us to tailoring our selection to those we could test in both WebAssembly and C languages



Summary of Results / Conclusions to Date

- **CWE-121: Stack-based Buffer Overflow:**
WebAssembly environment lacks the detection of stack-based buffer overflow in certain conditions against native environment.
- **CWE-122: Heap-based Buffer Overflow:**
WebAssembly environment has the higher risk to get exploitation by heap-based buffer overflow.
- **CWE-476: NULL Pointer Dereference:**
C++'s object oriented design allows for undefined behavior to be undetected in certain circumstances



Issues / Challenges

- Adapting complex system software for testing with new languages.
- The response time of remote virtual machine is slow than expected.
- Mitigation for Issues:
 - Early pilot testing.
 - Expert consultations.
- Iterative refinement of software test cases.
- Ensuring the accuracy of vulnerability comparisons.
- Team members' inexperience with the new languages.
- Some team members lack a computer science background.
- Mitigation for team challenges:
 - Delegating tasks to group members based on their background.
 - Utilizing online research to learn about the software in use.



Bibliography

- [1] Daniel Lehmann, Johannes Kinder, Michael Pradel, “Everything Old is New Again: Binary Security of WebAssembly,” in the Proceedings of the 29th USENIX Security Symposium, 2020.
- [2] C. Disselkoen, J. Renner, C. Watt, T. Garfinkel, A. Levy, and D. Stefan, “Position paper: Progressive memory safety for webassembly,” in Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy, ser. HASP '19. New York, NY, USA: Association for Computing Machinery, 2019.
- [3] Quentin Stiévenart, Coen De Roover, “Mohammad Ghafari The Security Risk of Lacking Compiler Protection in WebAssembly,” in International Conference on Software Quality, Reliability and Security, 2021.
- [4] Suhyeon Song, Seonghwan Park, Donghyun Kwon, “metaSafer: A Technique to Detect Heap Metadata Corruption in WebAssembly,” in IEEE Access, 2023.
- [5] Minseo Kim, Hyerean Jang, Young-joo Shin, “Avengers, Assemble! Survey of WebAssembly Security Solutions,” in IEEE International Conference on Cloud Computing, 2022.
- [6] Marius Musch, Christian Wressnegger, Martin Johns, Konrad Rieck, “New Kid on the Web: A Study on the Prevalence of WebAssembly in the Wild,” in International Conference on Detection of intrusions and malware, and vulnerability assessment, 2019.



Acronyms

- **AI:** Artificial Intelligence
- **CWE:** Common Weakness Enumeration
- **CTF:** Capture The Flag (inferred context, not explicitly defined)
- **EMCC:** Emscripten Compiler Frontend (inferred context, not explicitly defined)
- **GDB:** GNU Debugger
- **GCC:** GNU Compiler Collection (inferred context, not explicitly defined)
- **NSA:** National Security Agency
- **NVD:** National Vulnerability Database
- **PIT:** Platform IT
- **Wasm:** WebAssembly



Terms / Definitions

- **CWE (Common Weakness Enumeration):** A categorization of software weaknesses and vulnerabilities.
- **GDB (GNU Debugger):** A debugger offers extensive facilities for tracing and altering the execution of computer programs.
- **NSA (National Security Agency):** A U.S. agency responsible for national security and cryptography.
- **NVD (National Vulnerability Database):** A database that contains security checklist references, security-related software flaws, and misconfigurations.
- **PIT (Platform IT): InforAI (Artificial Intelligence):** Refers to the simulation of human intelligence in machines that are programmed to think like humans and mimic their actions. AI can be applied to various sectors including software development for tasks such as vulnerability detection and remediation.
- **CWE (Common Weakness Enumeration):** A list that categorizes common software weaknesses and vulnerabilities. It serves as a standard for identifying, mitigating, and preventing software vulnerabilities.
- **CTF (Capture The Flag):** In cybersecurity, it refers to a competition where participants solve security-related challenges to find a specific piece of data, called the "flag." This term was inferred and not explicitly defined in the provided text.
- **EMCC (Emscripten Compiler Frontend):** A tool used to compile C and C++ code into WebAssembly. It allows developers to run C and C++ code on the web at near-native speed.
- **GDB (GNU Debugger):** A debugger that provides extensive facilities for tracing and altering the execution of computer programs. It's part of the GNU Project and supports various programming languages including C and C++.
- **GCC (GNU Compiler Collection):** A compiler system produced by the GNU Project supporting various programming languages. It's standard for compiling C and C++ code, among others.
- **NSA (National Security Agency):** A U.S. government agency responsible for national security and the protection of government communications and information systems against penetration and network warfare.
- **NVD (National Vulnerability Database):** A database that contains information about security-related software flaws, misconfigurations, product names, and impact metrics. It's managed by the National Institute of Standards and Technology (NIST).
- **PIT (Platform IT):** Refers to information technology solutions and systems that support specific platforms or operations, often in a specialized or embedded context. This term was inferred and not explicitly defined in the provided text.
- **Wasm/WebAssembly:** A binary instruction format for a stack-based virtual machine. WebAssembly is designed as a portable compilation target for high-level languages like C/C++ and Rust, enabling deployment on the web for client and server applications.